Phase 5: Orders, Positions & Live P&L

What we built

- An **order panel**: Buy/Sell, **Market/Limit**, price and quantity inputs, 25/50/75/100% buttons, the order total and your available balance
- **Validation with Binance's real rules**: tick size, step size, minimum quantity, minimum order value, enough cash, enough coins
- A **simulated account** (paper trading): 10,000 USDT to start, saved in `localStorage` so it survives a refresh
- **Limit orders** that wait and **fill automatically** when the live price reaches them, plus a Cancel button
- A **Portfolio panel**: Equity / Cash / Unrealized / Realized P&L, a **positions table with live P&L**, open orders, and history
- The money math as **pure functions**, ready for unit tests in Phase 9

Files and what each one does

File	Role
`src/trading/portfolio.ts`	**Pure P&L math**: `applyFill` (average price, realized P&L), `unrealizedPnl`, totals, reservations
`src/trading/validation.ts`	**Pure validation**: `validateOrder()` → errors per field. Also `isMultipleOf`, `floorToStep`
`src/store/tradingStore.ts`	Second Zustand store = our **fake exchange**: place / fill / cancel / match orders, with `persist`
`src/components/OrderForm.tsx`	The order ticket: controlled inputs and live validation
`src/components/PortfolioPanel.tsx`	Account summary + Positions / Open orders / History tabs
`src/config.ts`	`SYMBOL_RULES` (Binance's real filters) and `STARTING_CASH`

How an order flows

```
OrderForm (strings typed by the user)
   │ validateOrder() on every render → errors shown live under each field
   │ submit
   ▼
tradingStore.placeOrder(draft)
   │ validateOrder() AGAIN (the "exchange" never trusts the UI)
   ├── MARKET, or a LIMIT already at a good price → fill now at the live price
   └── otherwise LIMIT → openOrders (its cash/coins become "reserved")
                              │
     every animation frame: useMarketFeed → matchOrders(batch)
                              │ BUY: price ≤ limit?  SELL: price ≥ limit?
                              ▼
                       fill at the limit price → cash, position, history updated
```

Concepts (theory only)

1. Market vs Limit orders

	Market	**Limit**
You say	"Buy now, whatever the price"	"Buy only at ≤ X" (or sell only at ≥ X)
Fills	Immediately (if there are sellers)	When the market reaches your price, maybe never
Risk	**Slippage**: the price you get can be worse than the one you saw	**No fill**: the price may never come
Role	**Taker** (removes liquidity from the book)	**Maker** while it waits in the book (adds liquidity)

A limit order whose price is *already* reachable (a buy limit above the current price) fills immediately, like a market order. Real exchanges do that too. Our simulation does it on placement. **Simplifications** (say these if asked): we fill at the **last traded price**, while a real market buy fills against the **best ask** in the order book (Phase 6), possibly across several levels. There are no fees and no partial fills.

2. Controlled inputs & forms

- **Controlled input:** `<input value={state} onChange={e => setState(e.target.value)}>`. React state is the single source of truth, and the input just displays it.
- Inputs hold **strings**: halfway through typing `0.05` the value is `"0."`, which isn't a useful number yet. We parse when validating.
- `type="text" inputMode="decimal"` + a regex filter instead of `type="number"`: number inputs accept `e`, change value when you scroll over them, and are awkward to control.

- `<form onSubmit>` + `e.preventDefault()`: Enter submits, and the browser doesn't reload the page. Buttons that shouldn't submit need `type="button"`.
- **`key={symbol}` resets a form**: a new key remounts the component, so all of its state starts fresh for the new coin.
- **Errors are derived, not stored:** `validateOrder` runs on every render, so there's no `errors` state to forget to update. We show an error only once the field has been typed in, or after a submit attempt (don't yell at users before they start).

3. Validation rules (real Binance filters)

Rule	Binance filter	BTCUSDT value	Why exchanges have it
Price must be a multiple of the **tick size**	`PRICE_FILTER`	0.01	Keeps the order book in discrete price levels
Quantity must be a multiple of the **step size**	`LOT_SIZE`	0.00001 BTC	Same, for amounts
Quantity ≥ **min qty**	`LOT_SIZE`	0.00001 BTC	
Price × qty ≥ **min notional**	`NOTIONAL`	5 USDT	Stops "dust" orders that cost more to process than they're worth
Enough **available** USDT (buy) / coins (sell)	n/a (balance check)		Spot trading = no borrowing, so you can only sell what you own

We copied the values from `GET /api/v3/exchangeInfo`. A production app would fetch them at startup. **Validate on both sides:** the form validates for good UX, and the "exchange" (our store) validates again, because the UI can be stale or bypassed. On a real system, **the server is the authority**.

4. Floating-point money

`0.1 + 0.2 = 0.30000000000000004` in JavaScript (and in Java's `double`). Consequences: - `x % step === 0` is unreliable, so `isMultipleOf` divides and checks "close to a whole number". - Quantities drift (`0.1 + 0.2 − 0.3 ≠ 0`), so we `clean()` them by rounding to 10 decimals. - Production systems use **integers in the smallest unit** (satoshis/cents) or a decimal library, like Java's **`BigDecimal`**. Binance itself sends prices as strings for the same reason (Phase 2).

5. Positions, average price, realized vs unrealized P&L

- **Position** = how much of a coin you hold + your **average entry price** (the cost basis).
- **Buying more** changes the average: `newAvg = (oldQty × oldAvg + buyQty × buyPrice) / (oldQty + buyQty)` e.g. 1 @ 100, then 1 @ 200 → 2 @ **150**
- **Selling** does **not** change the average of what's left. It **realizes** P&L on the part sold: `realized += (sellPrice − avg) × sellQty`. e.g. sell 1 @ 180 → **+30 realized**, still 1 @ 150
- **Unrealized P&L** = paper profit on what you still hold: `(markPrice − avg) × qty`. It moves with every tick. "**Mark price**" = the price used to value a position (we use the last trade price).

- **Equity** = cash + market value of all holdings. **P&L %** = `markPrice / avg − 1`.
- This is the **weighted-average cost** method. Others exist (FIFO: sell the oldest lots first), which give different realized numbers.

6. Reserved balances

An open limit buy for 0.1 BTC @ 60,000 has **promised** 6,000 USDT. If you could spend it again, both orders could fill and you'd go negative. So **available = balance − reserved by open orders**. Cancel the order and the reservation disappears. (On Binance you see this as "Available" vs "In order".) We don't store the reserved amount; we **derive** it from `openOrders`, so it can never get out of sync.

7. Pure functions + a thin store

The math (`applyFill`, `unrealizedPnl`, `validateOrder`) has no React and no store inside. The store just **calls** them and saves the results. Benefits: one place per formula, trivial unit tests, and you can explain each formula on a whiteboard.

8. More store techniques

- **Two stores** (market data vs account) with separate concerns. `tradingStore` reads prices with `useMarketStore.getState()` (no subscription needed outside components).
- **`persist` middleware**: saves chosen fields (`partialize`) to `localStorage` under a key, and reloads them on startup.
- **Returning the same state from `set`** means "nothing changed": Zustand skips notifying anyone. `matchOrders` does that on most frames, when no order fills.
- **Matching runs inside the rAF batch** (Phase 3): at most once per frame, with the latest prices.
- **Fine-grained subscriptions again:** each `PositionRow` subscribes to its own coin's price, so a BTC tick re-renders only the BTC row's P&L.

Try it yourself

1. **Market buy:** Buy → Market → click 25% → Buy BTC. Check Positions: avg price, live mark, P&L flickering green/red.
2. **Average price:** buy BTC again a minute later. The average moves between the two prices.
3. **Limit order that waits:** Limit → set a price slightly *below* the current one → Buy. It appears in **Open orders**, and "Available" USDT drops (reserved). When the price dips to it, it fills and moves to History.
4. **Cancel** a limit order: the reserved USDT comes back.
5. **Validation:** type `0.000001` BTC (step error), `0.00001` (under 5 USDT), or 100% then a bit more ("Not enough USDT"). Switch to Sell with no BTC ("More than you hold").
6. **Realized P&L:** click **Close** on a position. Unrealized turns into Realized in the summary.
7. **Refresh the page:** the account is still there (`localStorage`). DevTools → Application → Local Storage → `trading-dashboard-account`.

Resume mapping

> “**order screen** supporting **Market/Limit** orders with client-side **validation**” → `OrderForm.tsx` + `validation.ts` “**positions** with **live P&L**” → `PortfolioPanel.tsx` + `portfolio.ts` Be honest: **orders are simulated** (paper trading) against live prices. Real orders need API keys and a backend.

Interview Qs

1. **Market vs limit order?** Market fills now at the best available price, with slippage risk. Limit fills only at your price or better, and might never fill. A marketable limit fills immediately.
2. **How do you compute average price?** Weighted by quantity: total cost ÷ total quantity. Buys update it. Sells don’t, they realize P&L.
3. **Realized vs unrealized P&L?** Realized is locked in by selling: (sell – avg) × qty sold. Unrealized is paper P&L on holdings: (mark – avg) × qty, and it updates live.
4. **What validation does your order form do?** Tick size, step size, min qty, min notional (Binance’s real filters), and available balance net of open orders. It runs live in the form and again in the store before execution.
5. **Why validate twice?** The UI can be stale or bypassed. The executing side must enforce the rules. In production that’s the server.
6. **Why are inputs strings?** Intermediate states like “0.” aren’t valid numbers. We parse during validation.
7. **How do you avoid floating-point problems?** Tolerance-based multiple checks and rounding in this demo. In production: integer minor units or a decimal type (BigDecimal), and APIs that send numbers as strings.
8. **How do limit orders fill in your app?** On every price batch (once per frame), `matchOrders` checks each open order: a buy fills if price ≤ limit, a sell if price ≥ limit, at the limit price.
9. **Why do limit orders reduce “available” balance?** The cash/coins are reserved so they can’t be committed twice. Available is derived as balance – reserved.
10. **How does the account survive a refresh?** Zustand’s `persist` middleware serializes the data part of the store to localStorage.